Python Day 19

# Python 90 Days Challenge

# Chapter 11. Advanced Functions

Advanced functions help us write more flexible, reusable, and compact code.

Before starting, remember:

In Python, functions are objects. This means:

1. A function can be stored in a variable.
2. A function can be passed as an argument.
3. A function can be returned from another function.
4. A function can be written inside another function.

# 11.1 Functions as First-Class Objects

## Syntax

```
def function_name():
    statement

new_name = function_name
```

In Python, functions behave like normal objects. We can store a function in a variable and call it using that variable.

## Flow Chart

Create function
  |
  v
Assign function to variable
  |
  v
Call function using new variable

## Example 1

PYTHON CODE

```
def greet():
    print("Hello Python")
message = greet
message()
```

Output:
Hello Python
Here, message refers to the same function as greet.

CodeWithNishchal

# 11.2 Lambda Functions

A **lambda function** is a small anonymous function. Anonymous means it does not need a normal function name.

## Syntax

**lambda arguments: expression**

1. Lambda functions are used for small one-line functions.
2. They can take any number of arguments.
3. They can contain only one expression.
4. They automatically return the result of the expression.
5. Lambda functions are commonly used with map(), filter(), and sorting.

## Example 1

**Normal function:**

```python
def square(number):
    return number * number

print(square(5))
```

**Output:**
25
Same logic using lambda:

```python
square = lambda number: number * number
```

**Output:** 25

## Example 2

```python
add = lambda a, b: a + b

print(add(10, 20))
```

**Output:**30

Lambda is best for simple logic. For complex logic, use normal def functions.

CodeWithNishchal

# 11.3 map()

map() applies a function to every item of an iterable.

## Syntax

map(function, iterable)

## Explanation

1. map() takes a function and an iterable.
2. It applies the function to each item.
3. In Python 3, map() returns a map object, which is an iterator.
4. To display all results at once, convert it using list().

## Example 1

```
PYTHON CODE
numbers = [1, 2, 3, 4]
squares = map(lambda number: number * number, numbers)
print(list(squares))
```

**Output:** [1, 4, 9, 16]

## Example 2

```
names = ["aman", "riya", "kabir"]
upper_names = map(str.upper, names)
print(list(upper_names))
```

**Output:** ['AMAN', 'RIYA', 'KABIR']

## map() Table

| Part | Meaning |
|---|---|
| lambda number: number * number | Function to apply |
| numbers | Iterable |
| map() | Applies function to every item |
| list() | Converts result into a list |

CodeWithPanchal

# 11.4 filter()

filter() is used to select items from an iterable based on a condition.

## Syntax

filter(function, iterable)

## Explanation

1. filter() takes a function and an iterable.
2. The function must return True or False.
3. Items that return True are kept.
4. Items that return False are removed.
5. In Python 3, filter() returns a filter object, which is an iterator.

## Flow Chart

Iterable values
  |
  v
filter() checks condition
  |
  ├── True  -> keep item
  |
  └── False -> remove item
  |
  v
Filtered result

## Example 1

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(lambda number: number % 2 == 0, numbers)
print(list(even_numbers))
```

**Output:** [2, 4, 6]

## map() vs filter()

| Point | map() | filter() |
|---|---|---|
| Purpose | Transforms every item | Selects matching items |
| Output length | Usually same as input | Can be smaller |
| Function returns | New value | True or False |
| Example use | Square every number | Keep even numbers |

CodeWithNishchal

# 11.5 reduce()

reduce() combines all items of an iterable into a single final value. reduce() is not directly available like map() and filter(). It must be imported from functools.

## Syntax

from functools import reduce

reduce(function, iterable)

## Explanation

1. reduce() takes a function and an iterable.
2. It combines values step by step.
3. It returns one final result.
4. It is useful for cumulative calculations.
5. For simple addition, sum() is usually better.

## Example 1

PYTHON CODE

```
from functools import reduce
numbers = [1, 2, 3, 4]
total = reduce(lambda a, b: a + b, numbers)
print(total)
```

**Output:** 10
Working idea:
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10

## map() vs filter() vs reduce()

| Function | Purpose | Final Result |
|---|---|---|
| map() | Transform items | Iterator of transformed items |
| filter() | Select items | Iterator of selected items |
| reduce() | Combine items | Single value |

Code Nishchal